

Certifiable Support Budgets versus Intrinsic Support in Quantum Compilation

Sze Chun Yiu
Independent Researcher
`sze-chun.yiu@fysik.su.se`

Abstract

A support ceiling can be exact for a restricted proof system and still overestimate the support intrinsically required by the compiler model it abstracts. We formalize this distinction for a rank-only zero-sum deletion calculus. Its states are nonzero-total binary signature words, and its sole rule deletes a zero-sum subsequence. Terminal words have length at most the rank of the alphabet's generated span; a basis word is a matching countermodel to every smaller uniform conclusion. When the deletion is sound for a fixed compiler objective, this operational budget upper-bounds intrinsic optimal support. For a one-Tag, three-block family F_M under its frozen objective C_M , the rank-only frame-support budget and intrinsic frame support both equal two; the lower direction is witnessed by a feasible two-site support-two state of cost 5 versus a complete support-at-most-one optimum of 6. For a dependent-triple family F_I under its frozen objective C_I , each rank-only word position represents a block column in which at least one independent frame is active. The resulting joint active-column budget is five, whereas a whole-system Tag relocation gives intrinsic joint active-column support one. The five-to-one separation is only relative to this declared state language and this common support statistic. It is not a lower bound against richer proof systems and implies no runtime or hardware advantage.

Keywords: quantum compilation; certifiable support; intrinsic support; proof systems; Pauli strings

1. Scientific question and contribution

Support bounds appear at several distinct layers of a compiler argument:

1. the smallest support attained by an exact optimum;
2. the support reached by a named normalization; and
3. the support that a restricted proof system can certify.

Conflating these layers can turn a limitation of the proof language into a false compiler lower bound. The distinction is especially relevant for Pauli-string compilation, where block-wise compiler transformations can exploit structure that is invisible to coordinatewise signatures [1,2].

The paper's contribution is a pair of within-family comparisons, each keeping both the compiler objective and the reported support statistic fixed between the certificate and intrinsic quantities. In the matched family F_M , a rank-only frame-support certificate and the intrinsic maximum frame support both equal two under C_M . In the separated family F_I , the rank-only joint active-column budget is five under C_I , while the intrinsic joint active-column support is one because the compiler can relocate and reconstruct shared Tags at whole-system scope. The gap identifies information missing from the certificate language.

The rank-only system is deliberately restricted. It is not claimed to model every proof method used in practice, and no lower bound is asserted for every local, syndrome-preserving, or unrestricted proof system. Zero-sum sequence theory, general proof-system relativity, and direct-product arithmetic are prior or elementary scaffolds [3-5].

2. Intrinsic support

Let F be a family of finite exact-compilation instances. For an instance I , let $X(I)$ be its feasible set, $C_I : X(I) \rightarrow \mathbb{R}$ its fixed objective, and $\sigma_I(x)$ the support statistic under study.

Definition 1 (intrinsic optimal support). Define

$$\kappa(I; C) = \min\{\sigma_I(x) : x \in \arg \min_{y \in X(I)} C_I(y)\},$$

and, when finite,

$$\kappa(F; C) = \sup_{I \in F} \kappa(I; C).$$

This is a mathematical property of the family and objective. An upper theorem and a lower witness establish its exact value; they are not part of the definition.

The second quantity is deliberately proof-system-relative. To make its lower-bound obligation checkable, we define the proof system operationally rather than referring to an unspecified notion of what it can prove.

3. Rank-only zero-sum deletion calculus

Let $A \subseteq \mathbb{F}_2^d$ be an alphabet fixed before optimization and let $H = \langle A \rangle$. Write

$$\mathcal{L}(A) = \{(v_1, \dots, v_m) \in A^* : v_1 \oplus \dots \oplus v_m \neq 0\}.$$

A subsequence selects an arbitrary set of positions and need not be contiguous. The rank-only calculus $P_{\text{rank}}(A)$ has exactly one transition:

$$(v_1, \dots, v_m) \longrightarrow (v_i)_{i \notin Q} \quad \text{when} \quad \emptyset \neq Q \subseteq \{1, \dots, m\}, \quad \bigoplus_{i \in Q} v_i = 0.$$

Deleting a zero-sum subsequence preserves the nonzero total, so the deleted set cannot be the whole word and the successor remains in $\mathcal{L}(A)$. A word is terminal precisely when it is zero-sum-free. For $w \in \mathcal{L}(A)$, define its best rank-only normal-form length by

$$\nu_A(w) = \min\{|u| : w \longrightarrow^* u \text{ and } u \text{ is terminal}\},$$

and define the **rank-only certifiable support budget**

$$\beta_{\text{rank}}(A) = \sup(\{0\} \cup \{\nu_A(w) : w \in \mathcal{L}(A)\}).$$

The identity derivation is allowed, and every nonidentity transition strictly shortens the word, so the minimum exists. The term *certifiable support budget* is used to avoid collision with the established Boolean-function quantity called certificate complexity.

For a compiler family F , each constrained object G has a fixed alphabet $A_{I,G}$, and one word position represents one unit of the support statistic assigned to G . Once the deletion rule has been proved sound for the objective C , set

$$\beta_{P_{\text{rank}}}(F; C) = \sup_{I \in F, G} \beta_{\text{rank}}(A_{I,G}).$$

Here C records the soundness binding. It does not add an unlisted inference rule to the rank-only calculus.

Proposition 1 (compiler soundness). Suppose every admitted instance has an exact C -optimum, and every rank-only transition on the signature word of a C -optimal compiler state is realized by a feasible C -optimal compiler state whose support for the constrained object is the successor-word length. Then

$$\kappa(F; C) \leq \beta_{P_{\text{rank}}}(F; C).$$

Proof. Start from a C -optimum and apply rank-only transitions until a terminal word is reached. Strict length descent gives termination, the hypothesis preserves feasibility and optimality, and the terminal support is at most the declared family budget. Taking the supremum over instances gives the inequality. $\square$

The reverse inequality does not follow. The calculus may omit a valid compiler transformation.

Theorem 2 (exact rank-only budget). For every finite $A \subseteq \mathbb{F}_2^d$,

$$\beta_{\text{rank}}(A) = \text{rank}(\langle A \rangle).$$

Proof. A terminal word is zero-sum-free, hence its listed vectors are linearly independent and its length is at most $r = \text{rank}(H)$. Repeated deletion therefore gives $\nu_A(w) \leq r$ for every $w \in \mathcal{L}(A)$. If $r = 0$, then $\mathcal{L}(A)$ is empty and the explicit zero in the definition gives $\beta_{\text{rank}}(A) = 0$. If $r > 0$, then because A spans H , it contains a nonempty basis B . The word listing B has nonzero total and no nonempty zero-sum subsequence. It is terminal, so its only reachable word is itself and $\nu_A(B) = r$. Thus no conclusion below r holds in the declared state language, and the two bounds coincide. $\square$

The basis word is a countermodel to a smaller conclusion from the rank-only premises. It is not automatically a lower bound for the compiler. A compiler may have a smaller optimum because it uses structure that the alphabet model discards, and a richer proof system may formalize that structure.

4. Fixed objective and family definitions

We use phase-quotiented n -qubit Paulis $P = (x, z) \in \mathbb{F}_2^n \times \mathbb{F}_2^n$. Multiplication is componentwise bitwise exclusive-or (XOR), $w(P)$ counts coordinates whose local letter is nonidentity, and

$$\langle (x, z), (x', z') \rangle = x \cdot z' + z \cdot x' \pmod{2}.$$

The local three-way Restore functional is

$$f_3(a, b, c) = \begin{cases} 1, & a = b = c \neq I, \\ \mathbf{1}[a \neq I] + \mathbf{1}[b \neq I] + \mathbf{1}[c \neq I], & \text{otherwise.} \end{cases}$$

4.1 Matched three-block family

An instance of F_M supplies three ordered target pairs $(T_{\ell 0}, T_{\ell 1})$, with $\ell \in \{A, B, C\}$. A compiler state contains six frame Paulis $R_{\ell k}$, one shared Tag S , a central role $c_\ell \in \{0, 1\}$ for each block, and an allowed within-pair target order π_ℓ . It is feasible when

$$\langle R_{\ell 0}, R_{\ell 1} \rangle = 1 \quad \text{for every } \ell,$$

and the two Tag labels are block-independent and distinct:

$$\langle S, R_{Ak} \rangle = \langle S, R_{Bk} \rangle = \langle S, R_{Ck} \rangle = \lambda_k, \quad \lambda_0 \neq \lambda_1.$$

Set $m_{\ell k} = 2$ when $k = c_\ell$ and $m_{\ell k} = 4$ otherwise. The frozen unit objective is

$$\begin{aligned} C_M = & \sum_{\ell, k} m_{\ell k} w(R_{\ell k}) + 2w(S) - 18 \\ & + \sum_{q=1}^n \sum_{k=0}^1 f_3((T_{Ak}^{\pi_A} R_{Ak})_q, (T_{Bk}^{\pi_B} R_{Bk})_q, (T_{Ck}^{\pi_C} R_{Ck})_q). \end{aligned}$$

The support statistic is $\sigma_M(x) = \max_{\ell, k} w(R_{\ell k})$. Each constrained object is one frame, and each word position is one active coordinate of that frame. Its signature records the symplectic bit with its partner and with S , so the rank-only span has dimension at most two.

4.2 Separated dependent-triple family

An instance of F_I has two blocks $j \in \{A, B\}$. Each block contains independent frames R_{j0}, R_{j1} with $\langle R_{j0}, R_{j1} \rangle = 1$ and dependent frame $R_{j2} = R_{j0}R_{j1}$. For a transparent canonical-label subfamily, two shared Tags satisfy, in both blocks,

$$\begin{aligned} (\langle S_0, R_{j0} \rangle, \langle S_0, R_{j1} \rangle) &= (0, 1), \\ (\langle S_1, R_{j0} \rangle, \langle S_1, R_{j1} \rangle) &= (1, 0). \end{aligned}$$

The broader compiler acceptance rule allows any ordered pair of distinct nonzero two-bit Tag-label vectors. The fixed pair above defines the narrower family used in this paper; the parent normalization covers it, and no claim about the broader label family is needed below.

For target triples T_{jk} , an allowed target permutation, and central role $c_j \in \{0, 1, 2\}$, let $m_{jk} = 2$ for $k = c_j$ and $m_{jk} = 4$ otherwise. The frozen unit objective is

$$C_I = \sum_j \left[\sum_{k=0}^2 m_{jk} (w(R_{jk}) - 1) + \sum_{k=0}^2 w(T_{jk} R_{jk}) \right] + 2(w(S_0) + w(S_1)).$$

For block j , define its active-column set and the family statistic by

$$U_j(x) = \text{supp}(R_{j0}) \cup \text{supp}(R_{j1}), \quad \sigma_I(x) = \max_{j \in \{A, B\}} |U_j(x)|.$$

This joint statistic is essential. One rank-only word position represents one column $q \in U_j(x)$, and its symbol is the parity-change vector induced by simultaneously replacing $(R_{j0, q}, R_{j1, q})$ with (I, I) ; the dependent letter $R_{j2, q}$ then also becomes I . The production transition is recorded in ten binary coordinates. For either block, its changes lie in and span a five-dimensional subspace, which is the alphabet used by the rank-only abstraction.

A zero-sum set of these block-column symbols preserves all five block parities. Objective soundness is a separate obligation: the exact local analysis covers all 46,080 active-letter, target-letter, Tag-letter, and central-role rows and finds that the new local contribution to C_I is at least four units smaller for every deleted active column. Because the objective is coordinate-additive and the Tags are held fixed, a simultaneous zero-sum deletion is therefore feasible and objective-nonincreasing. The compiler semantics additionally permit each dependent-triple block to be localized to one anticommuting core and the shared Tags to be reconstructed at whole-system scope.

The objectives C_M and C_I are different fixed formal objectives. Every comparison below keeps the relevant objective and support statistic fixed between β and κ . Neither rank-only language is claimed to be a complete move registry for an external production implementation.

5. Matched family: certificate and intrinsic support coincide

In F_M , the frame alphabet spans two binary signature coordinates and admits a basis word. Theorem 2 gives

$$\beta_{P_{\text{rank}}}(F_M; C_M) = 2.$$

Separately, the whole-instance normalization gives an exact optimum of support at most two for every admitted size. The lower direction can be checked on the following two-site instance. Pauli keys are the binary masks (x, z) defined in Section 4. Each displayed integer is the decimal encoding of a two-bit mask, so $(0, 2)$ and $(3, 0)$ denote two-qubit Paulis rather than individual coordinates.

Block	Target pair $(T_{\ell 0}, T_{\ell 1})$	Feasible support-two frame pair	A minimizing support-at-most-one frame pair
A	$((0, 1), (0, 1))$	$((0, 1), (1, 1))$	$((0, 1), (1, 0))$
B	$((0, 1), (0, 1))$	$((0, 1), (1, 1))$	$((0, 1), (1, 0))$
C	$((0, 2), (2, 0))$	$((0, 2), (3, 0))$	$((0, 1), (1, 1))$

Both displayed states use $S = (0, 1)$. For the support-two state, the central choices are $(0, 0, 1)$ and the target orders are unchanged. Its objective components are

State	Weighted frame	Tag	Constant	Restore	C_M
displayed support-two state	20	2	-18	1	5
displayed support-at-most-one state	18	2	-18	4	6

The second row is not merely an example. The independent checker enumerates all $7^6 = 117,649$ six-frame tuples with frame support at most one, including all $12^3 = 1,728$ anticommuting frame tuples, all 16 Tags, all eight central choices, and all four allowed relative target orders. Its exact minimum is 6. The feasible support-two state costs 5, so no support-at-most-one state can be optimal for this instance. Therefore support one is not a uniform optimum bound and

$$\kappa(F_M; C_M) = 2.$$

Consequently

$$\beta_{P_{\text{rank}}}(F_M; C_M) = \kappa(F_M; C_M) = 2.$$

The rank-only certificate is therefore sharp for this family and objective. This does not imply sharpness for general MultiTag grammars.

6. Separated family: rank-only budget five, intrinsic support one

The enumerated block alphabets contain the following rank-five bases, written as integer encodings of their ambient 10-bit binary change vectors:

$$B_A = (1, 68, 136, 272, 544), \quad B_B = (2, 4, 8, 16, 32).$$

Each basis word is nonzero-total and zero-sum-free. The block word is built from active columns, not from the support positions of one selected frame. Since P_{rank} admits all nonzero-total words over the declared alphabet and has no rule other than zero-signature deletion, Theorem 2 gives the exact joint-column budget

$$\beta_{P_{\text{rank}}}(F_I; C_I) = 5.$$

The compiler can perform a transformation outside that language. It localizes both independent frames of each dependent-triple block to the same anticommuting core and then relocates or reconstructs the shared Tags. The fixed-objective exchange inequality pays for this transformation at every non-core coordinate. The all-size construction yields $|U_A| = |U_B| = 1$, while joint support zero is infeasible because two zero-supported frames cannot anticommute. Therefore

$$\kappa(F_I; C_I) = 1.$$

The exact separation is

$$\kappa(F_I; C_I) = 1 < 5 = \beta_{P_{\text{rank}}}(F_I; C_I).$$

Thus both sides of the inequality use the same statistic σ_I . The result diagnoses the operation missing from P_{rank} ; it does not show that every stronger local or unrestricted proof system needs budget five.

7. Declared product amplification

Let F_I^t be the direct product of t independent F_I components on disjoint coordinates, with product objective $C_I^{\oplus t}$ and support statistic $\sigma_I^{\oplus t}(x_1, \dots, x_t) = \sum_{h=1}^t \sigma_I(x_h)$. By definition, objectives and these component joint-column supports add, compiler moves remain componentwise, and the product proof system has no cross-component inference rule.

Proposition 3 (componentwise product budgets). For every integer $t \geq 1$,

$$\beta_{P_{\text{rank}}}(F_I^t; C_I^{\oplus t}) = 5t, \quad \kappa(F_I^t; C_I^{\oplus t}) = t.$$

Proof. Componentwise constructions give the upper bounds. In the direct-sum signature space, the union of the t disjoint five-vector bases is a zero-sum-free word of length $5t$. The product calculus has no cross-component rule, so this word is terminal and supplies the matching rank-only lower bound. Intrinsically, every component needs joint active-column support at least one because

support zero is infeasible, while the one-core normalization acts independently in each component. Additivity of $\sigma_I^{\oplus t}$ gives the two equalities. $\square$

The additive gap is $4t$, while the ratio remains five. This is a definitional amplification of the one-component mechanism, not an independent compiler phenomenon.

7.1 Direct-enumeration corollary

For a direct enumerator over n candidate block columns with fixed local alphabet and fixed joint active-column budget B , the leading search volume is $\Theta(n^B)$. At fixed t , the declared product budgets therefore give volumes $\Theta(n^{5t})$ and $\Theta(n^t)$, with ratio $\Theta(n^{4t})$ as $n \rightarrow \infty$. The separate statement that the additive gap grows with t concerns a different limit. Neither statement is an algorithm-independent complexity lower bound.

8. Discussion and relation to prior work

Subset-conditioned zero-sum sequences provide the combinatorial deletion scaffold [3], while general sparse integer optimization supplies broader support context [4]. Cook and Reckhow’s theory of relative proof-system efficiency is a conceptual predecessor for separating a mathematical property from what a fixed calculus can establish [5]. The present quantities are not Boolean proof complexity, and the paper claims no general proof-complexity theorem.

Within quantum compilation, Tag-and-Restore Encoding (TARE) makes Tag selection an optimization opportunity [1], and block-wise approaches such as Paulihedral show why global compiler transformations can outperform termwise reasoning [2]. The bounded residual here is the pair of within-family, same-statistic comparisons: F_M , where rank-only frame certification is sharp under C_M , and F_I , where a whole-system Tag transformation crosses the rank-only joint-column boundary under C_I .

The separated result should not be read as an indictment of local reasoning in general. The proof language was chosen to isolate one inference pattern: signatures are summarized by binary rank and shortening is limited to zero-signature deletion. Exactness of its budget means exactness relative to those rules. The strict inequality shows only that those rules discard compiler structure needed to reach the intrinsic optimum.

9. Reproducibility

The accompanying source binds the exact objective, family definitions, basis obstructions, support-two lower witness for F_M , and dependent-triple local lemmas for F_I . A standard-library verifier exhausts 2,880 deletion rows, 6,912 core-alignment rows, 576 same-site rigidity rows, and 9,216 distinct-site Tag rows, then checks the composition arithmetic and the support-zero obstruction. A second checker verifies the abstract binary and product statements and distinguishes joint active-column support from maximum individual-frame weight.

These are package-local finite checks, not external replication. The all-size values of κ depend on the displayed analytic composition arguments and the bound exact witnesses. No production move-registry completeness follows.

An earlier finite production-realization study of the matched family rejected its proposed certificate. The declared optimizer values were corroborated, but the study did not establish a complete production move registry; a source-bound follow-up again left production transfer unestablished.

These are adverse transfer results, not failures of the formal support-budget comparison. Their exact machine terminals and immutable evidence remain in the accompanying provenance ledger.

A separate, prospectively ordered production-transfer attempt used pinned PyZX 0.10.5. It executed 74 of 4,681 ordered two-qubit circuit words before failing closed on the word consisting of three consecutive Hadamard gates on qubit 0. After a semantics-preserving prefix, the callable guard accepted a second boundary-pivot simplification, but that transition changed the dense linear map even up to a nonzero scalar. The scheduled production reduction preserved the map on the same word. This adverse result refutes only the proposed freely reorderable language of twelve macros under their callable guards. It does not refute the scheduled production reduction, establish complete PyZX move coverage, realize a certificate gap, or authorize a complete-domain null. The round is consumed as an indeterminate completeness study and is not retuned or relabelled; its exact machine-readable terminal is retained in the provenance ledger.

10. Limitations

The exact value of $\beta_{P_{\text{rank}}}$ is relative to the explicitly defined rank-only proof system. We prove no lower bound that applies to every local, syndrome-preserving, or unrestricted proof system. The reported values for F_M and F_I are likewise tied to the stated fixed objective, support statistic, and formal families. In particular, the F_I statistic is joint active-column support rather than maximum individual-frame weight.

The product theorem assumes additive joint-column support, an additive objective, and the absence of cross-component moves or inference rules. Its enumeration exponent therefore describes the direct block-column support enumerator, not arbitrary algorithms. More broadly, structural support is not gate count, depth, runtime, qubit count, error rate, or physical quantum advantage.

The production boundary remains unresolved. The adverse PyZX transfer record shows that callable macro guards omit load-bearing scheduler context, and the formal state languages are not established as complete move registries for a production compiler. The author-side verification reported here also does not constitute independent specialist proof review, novelty authority, peer review, or external replication.

11. Conclusion

Intrinsic optimal support and a proof system’s certifiable support budget answer different questions, but a numerical comparison is meaningful only when both use the same statistic. Soundness gives $\kappa \leq \beta$, while equality requires a compiler lower witness and strict inequality identifies structure missing from the proof language. The matched family F_M supplies the equality control at maximum frame support two. The separated family F_I has rank-only joint active-column budget five but intrinsic joint active-column support one because whole-system Tag reconstruction lies outside the certificate language. The result is deliberately proof-system-relative and carries no unrestricted complexity or hardware claim.

Data and code availability

The submission source contains the deterministic checkers, expected outputs, finite control records used in the manuscript, and the immutable adverse PyZX transfer record summarized above. No external dataset is used. These artifacts corroborate the stated finite case analyses and exact witnesses but do not establish external replication or production move completeness. The exact

source archive is available to editors and referees with the manuscript and will accompany the public manuscript record.

Author contributions

Sze Chun Yiu is the sole author and is responsible for the scientific claims and final manuscript. Generative AI tools assisted literature discovery, drafting, language editing, adversarial review, and submission-package preparation.

References

1. N. Schillo, A. Sturm, and R. Quay, “TARE: Block Encoding Linear Combinations of Pauli Strings Without Ancilla State Preparation,” arXiv:2601.05740v4 [quant-ph] (2026).
2. G. Li, A. Wu, Y. Shi, A. Javadi-Abhari, Y. Ding, and Y. Xie, “Paulihedral: A Generalized Block-Wise Compiler Optimization Framework for Quantum Simulation Kernels,” in *Proceedings of ASPLOS 2022*, 554–569 (2022), DOI.
3. A. Plagne and S. Tringali, “The Davenport Constant of a Box,” *Acta Arithmetica* **171**(3), 197–219 (2015), DOI.
4. I. Aliev, J. A. De Loera, F. Eisenbrand, T. Oertel, and R. Weismantel, “The Support of Integer Optimal Solutions,” *SIAM Journal on Optimization* **28**(3), 2152–2157 (2018), DOI.
5. S. A. Cook and R. A. Reckhow, “The Relative Efficiency of Propositional Proof Systems,” *Journal of Symbolic Logic* **44**(1), 36–50 (1979), DOI.